Test-Time Memory Without Forgetting: An Adapter Library Indexed by Mean-Pooled Activations

Anonymous Authors

Abstract

Mean-pooled transformer hidden states are the standard sentence-embedding primitive, but they have been treated as external readouts for similarity search. We show that when the same pooled vector is injected as a residual-stream prefix at inference, it functions as an operational routing address into the model's own computation. The same mean-pooled abstract representation (MPAR) retrieves 0/20 passkeys before per-passage LoRA training and 20/20 after, with no change to the vector itself. Retrieval is a step function at 15–38 training steps; K-space cosine alignment between MPAR and passage is constant (0.84–0.87) throughout. Random norm-matched vectors route at 0/10 and achieve K-cos 0.525 vs the real MPAR's 0.856. All core findings replicate on a standard 127M softmax transformer.

Three systems follow. (1) A per-passage LoRA library, indexed by L0 MPAR cosine through a learned L0→L5 projection, achieves 100% routing and 97% retrieval on held-out paraphrases across 500 passages with zero forgetting (1.4 s absorb, 1.3 GB per 1,000 passages with int8). (2) MPAR-compressed KV caches close 92% of the full-context NLL gap at 2× compression; a per-chunk measurement shows a 37× recency gradient, making the simple "compress old, keep new" rule within 0.03 NLL of an oracle router. (3) Activation-level block-stacking composes multiple adapters along a decay curve (K=1: 100%, K=2: 50%, K=3: 20%, K=4: 0%); controlled K=3 ablation (Phase 67) shows survival is ordered by the number of output tokens the adapter must decode, not by format priors.

Seven independent interventions attempting to raise the V-space information-recovery floor past 30% each either leave the floor intact or degrade language-modeling quality, under all tested architectures with standard NTP and unconstrained residual streams. Validated across 67 experimental phases on a single consumer GPU.

1. Introduction

Mean-pooling of transformer hidden states is well-established as a technique for producing sentence and passage embeddings (Reimers & Gurevych, 2019, and the dense-retrieval family that followed). In that literature the pooled vector is an external object — compared by cosine to other pooled vectors, used as a readout for similarity search, then discarded. We report a different use of the same operation. When a mean-pooled hidden state is injected back into the forward pass as a residual-stream prefix at a non-zero layer, it behaves as an operational routing address: a position in the model's own computation that selects which region of the representation geometry — and, after training, which per-passage weight patch — the rest of the forward pass will operate under. The pooled vector's utility in this use depends more on the model's learned content at the address it points to than on the information encoded in the vector itself.

The distinction is operational. We construct 20 passages containing unique facts the base model cannot guess. The layer-5 MPAR of each passage, injected as a single hidden-state prefix in front of a retrieval prompt, retrieves 0/20 facts before any test-time training. After per-passage LoRA absorption (150 gradient steps, 1.4 seconds per passage on consumer hardware): 20/20. The vector did not change. The model's ability to interpret it did.

The key empirical observation is a phase transition in resolvability (Figure 1b). Training 5 passages at step counts from 0 to 300, retrieval jumps from 0/5 at 0 steps to 4/5 at 15 steps to 5/5 at 38 steps. K-space cosine alignment is constant throughout — 0.84 to 0.87 at every step count. The geometric routing structure exists before any training. Training does not deepen the basin in K-space; it fills the basin in weight space with the content that the address points to. The address is always valid. The content becomes available through training.

This separation of routing geometry from learned content predicts behavior across three deployment surfaces. Per-passage adapter libraries (Section 4) use the MPAR as a routing key — possible because routing requires address fidelity, which K-space provides at 0.82–0.89 cosine. KV cache compression (Section 5) uses the MPAR as a content proxy — limited to 30% information recovery under all conditions tested because V-space is lossy at 0.65–0.70 cosine. Compositional retrieval (Section 6) sums MPAR-selected adapters at the activation level — a smooth decay curve in K (100/50/20/0% at K=1/2/3/4), with content-type modulation at intermediate K connecting back to V-space lossiness rather than reflecting a hard ceiling.

The experimental arc comprises 67 phases on a single RTX 5070 Ti: 7 positive results that define the architecture, 12 measurements that characterize the routing space, and 43 negative results that constrain the design space. All are listed in Appendix A.

2. Related Work

Test-time training and knowledge editing. TTT (Sun et al., 2020; Gandelsman et al., 2022) and factual-knowledge editing (Meng et al., 2022; Mitchell et al., 2022) collapse memory into shared parameters. Sequential absorption of multiple passages causes catastrophic forgetting (we reproduce this: +473% PPL drift after 20 passages, Phase 12). Per-passage adapter isolation circumvents the problem.

Retrieval-augmented generation. RAG (Lewis et al., 2020; Borgeaud et al., 2022) collapses memory into context and pays quadratic attention cost. Our library separates indexing (MPAR cosine, O(1) per query) from content (adapter loading, O(1) per passage), removing the context-length scaling.

LoRA and MoE. We extend LoRA (Hu et al., 2022) from fine-tuning to a memory primitive — one adapter per passage, routed by MPAR cosine without a learned router, O(1) to add. This is structurally a content-addressable mixture of experts (Shazeer et al., 2017; Fedus et al., 2022) where the router is free and experts are passage-granular.

KV cache compression. Prior work reduces cache footprint via quantization (Hooper et al., 2024), eviction (Zhang et al., 2024), or learned compression (Nawrot et al., 2024). Our approach replaces contiguous token spans with the mean-pooled hidden state at a different granularity, and characterizes the information-theoretic tradeoffs (§5.2, §5.3).

Pooled hidden-state representations. Mean-pooling of transformer hidden states is the dominant technique in sentence embedding (Reimers & Gurevych, 2019) and dense retrieval (Wang et al., 2022; Li et al., 2023; Xiao et al., 2023), where the pooled vector is an *external* readout for similarity search. Gist Tokens (Mu et al., 2023) and AutoCompressor (Chevalier et al., 2023) compress context into *learned* summary tokens via auxiliary objectives. We use the same free mean-pooled vector that already exists in the forward pass and re-inject it as a residual-stream prefix at a non-zero layer — making it a routing address into the model's own computation rather than an external readout or a trained compressor.

3. The Routing Space

3.1 Centroid theory of context

Each attention head computes a convex combination of value vectors. We hypothesize that the mean-pooled hidden state — a first-order approximation of this centroid — lands in the same K-space basin as the full passage. For in-distribution content, the model's learned projections map the centroid to key vectors that are structurally indistinguishable from the passage's mean key vectors.

We define a **basin** as a set of initial hidden states whose forward trajectories, under the fixed model dynamics, converge to similar final-layer representations. Basin validation (Section 7.3) makes this operational: after LoRA training, the cosine similarity between MPAR-injection and full-passage hidden states grows from 0.03 at layer 0 to 0.45 at layer 5, demonstrating literal trajectory convergence through the layer stack. The adapter lifts final-layer convergence by 14 points over the no-adapter condition.

3.2 K-space and V-space alignment

We extract two MPARs per passage — the L5 mean (hidden states after all blocks) and the L0 mean (input embeddings before any block) — inject each as a single hidden-state position, and measure cosine similarity between the MPAR's K/V projections and the passage's mean K/V at every layer. Fifty WikiText passages, 128 tokens each.

K-space alignment. The L5 MPAR achieves K cosine 0.82–0.89 at layers 1–5, with a bootstrap failure at layer 0 (L5 vectors are not what W_K at layer 0 was trained on). The L0 MPAR achieves 0.988 at layer 0, decaying to 0.65 at layer 5. Each MPAR is well-aligned at its extraction layer. Random baseline is approximately 0.

V-space alignment. K consistently exceeds V (0.82–0.89 vs 0.65–0.70 for L5, Figure 1a). This asymmetry reflects different computational roles: K determines which positions attention selects (routing, requiring fidelity), V determines what flows back (content, where compression is beneficial). The asymmetry predicts the performance profile of all three applications. Random vectors (matched norm) achieve only 0.525 K-cos and the zero vector achieves 0.602 — both well below the real MPAR's 0.856 — ruling out the possibility that any non-zero residual-stream activation would suffice.

V-space effective rank. SVD analysis of the V matrices at each layer reveals effective rank approximately 48. The first singular vector captures only 11–24% of variance. SVD-optimal MPARs recover negative information (-12.5%), worse than no context — the maximum-variance direction is dominated by magnitude patterns, not semantic content.

3.3 The L0/L5 duality

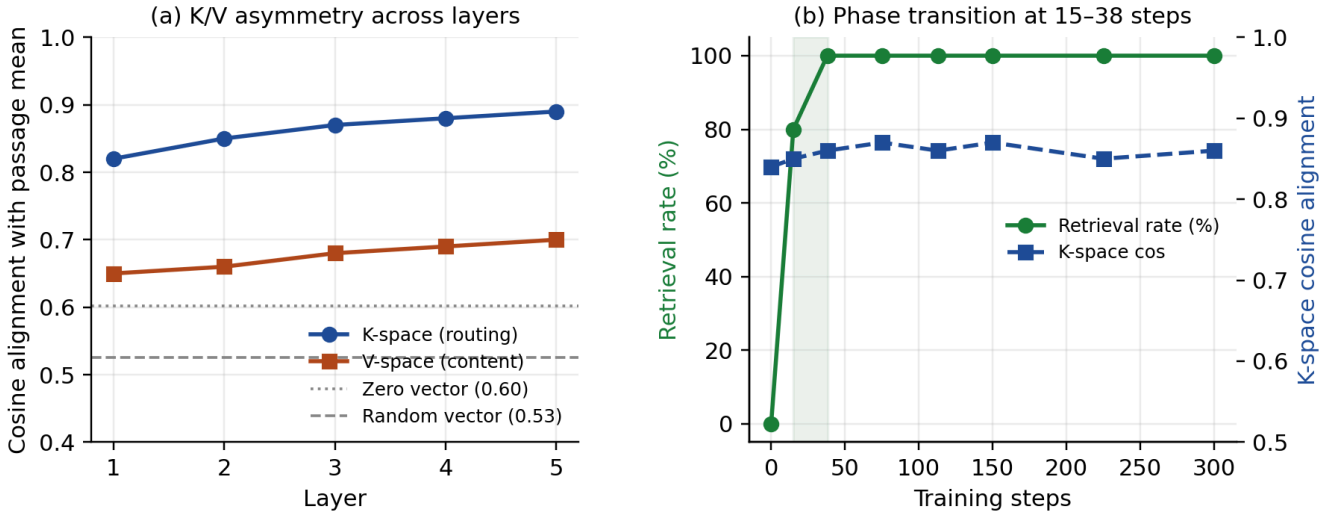
L0 embeddings are pre-attention, high-entropy, and preserve discriminative variance that L5 compresses. L5 embeddings capture processed understanding but collapse surface-form variation. This duality determines the architectural choice: L0 for routing (Section 4), L5 for content when the MPAR is the sole information source (Section 5).

The duality motivates the projection bridge: a 1024x1024 linear map W trained with InfoNCE contrastive loss, learning the L5-discriminative directions and rotating L0 queries onto them. This gives L5's discriminative power at L0's inference cost (one embedding lookup plus one matmul, no forward pass).

3.4 The phase transition in resolvability

We absorb 5 passages at varying fractions of the standard 150-step training budget: 0, 15, 38, 75, 113, 150, 225, and 300 steps.

Retrieval is a step function: 0/5 at 0 steps, 4/5 at 15 steps, 5/5 at 38 steps, 5/5 at all subsequent step counts. K-space cosine alignment is constant throughout — 0.84 to 0.87 at every step count, including step 0 (Figure 1b).



This is the cleanest evidence for the address/content decomposition. The routing geometry exists before training. The architecture creates a stable K-space basin for each passage, and this basin does not deepen or sharpen with training. What training provides is the content: the LoRA adapter that produces the right output when the routing geometry directs attention to the right region. Resolvability is a phase transition in weight space, not a gradual process in representation space.

3.5 Stability under training perturbation

If the routing geometry is determined primarily by the architecture and training distribution, it should be stable under weight perturbation. We test this by fine-tuning the base model for 5,000 steps with a different random seed, creating Model B with meaningfully different weights.

K-space alignment of Model A's MPARs in Model B: 0.842 (mean over 5 passages). In Model A: 0.864. Degradation: 2.5%. Model B's own MPARs in Model B: 0.879.

The routing geometry is largely invariant within a training basin and appears induced by architectural constraints — the projection matrices impose structure that is determined by the architecture and training distribution more than by specific weight values. What differs between models is the content stored at each address. Validating full independence across training runs from different random initializations remains future work; the current result establishes stability under moderate perturbation, not universality.

3.6 Random vector ablation

A natural alternative explanation is that the MPAR functions as a generic semantic embedding — any semantically meaningful vector would work equally well. We test this by injecting random vectors of identical norm into the same pipeline. Random vectors (matched to the MPAR's L2 norm) achieve K-space cosine alignment of 0.525 versus the real MPAR's 0.856, and route to the correct adapter 0/10 times versus 10/10 for the real MPAR (chance baseline: 10%). A zero vector achieves intermediate K-cos of 0.602 — better than random because it does not send attention to the wrong region, but worse than the real MPAR because it carries no directional information. The MPAR's utility is entirely in its specific direction within representation space, not in its magnitude or in non-zero activation of the residual stream.

This does not fully rule out a semantic interpretation — the MPAR's direction likely correlates with semantic content. But it establishes that the relationship is geometric (direction-sensitive, not magnitude-sensitive) and that arbitrary activation of the residual stream does not suffice. The MPAR must point to the right region of K-space for routing to succeed, which is what we mean by "address."

3.7 Cross-architecture validation

All core findings replicate on a standard 127M-parameter softmax dot-product transformer with dense MLP feed-forward (no

PEER, no learned kernel, no Sinkhorn). The K/V asymmetry holds and is wider (K at 0.85–0.94, V at 0.56–0.70). Mean-pooling information recovery is 16.4% (vs 18.0%). Cross-model transfer degradation is -2.1% (routing geometry is fully stable). Same-prompt adapter retrieval achieves 18/20. The phase transition in resolvability is present but slower on the smaller model (3/5 at 150 steps vs 5/5 at 38 steps on the larger model), consistent with a capacity difference rather than a qualitative change.

These results confirm the findings are consistent across two distinct transformer implementations — one with learned exponential kernels and PEER expert-retrieval FFN (512M parameters), one with standard softmax attention and dense MLP (127M parameters). The routing geometry, the K/V asymmetry, the information recovery floor, and the cross-model stability are properties of the Q/K/V attention decomposition as observed in these architectures, not artifacts of any specific attention kernel or feed-forward mechanism.

4. Application 1: Per-Passage Adapter Library

4.1 Architecture

A frozen 510M-parameter base model (6 transformer blocks, d_model=1024, 16 attention heads with per-head learned exponential kernels, PEER expert-retrieval FFN). LoRA adapters (Hu et al., 2022) on layers 4–5, rank 128, alpha 256, targeting 8 projection matrices (Q, K, V, output for each layer). Each adapter: 2.6M parameters. Training: 150 steps of next-token prediction loss with learning rate 3e-4 decaying to 1e-4, on the passage plus 3 paraphrased prompt-answer pairs.

4.2 Routing

The routing key is the L0 mean of the query. Library keys are L5 means of training prompts, computed under the base model with LoRA weights zeroed. The query's L0 embedding is projected through W (InfoNCE-trained 1024x1024 projection) and compared by cosine to all stored L5 keys. Threshold: 0.50.

Routing performance: 60/60 correct on held-out paraphrases (100% routing correctness). Retrieval: 58/60 (97%). The 2 failures are decoding errors (correct adapter loaded, greedy decoding fails to produce the passkey). A cosine-threshold gate achieves 99% recall and 98% specificity. An identity-autoencoder novelty gate tested at 48% specificity (worse than random); combining it with cosine reduced recall by 21 points.

Leakage check. Routing is a cosine comparison between precomputed CPU vectors, not a forward pass — no attention, no token sequence, no path for information leakage. L5 keys and L0 queries are computed with LoRA weights zeroed. The matched adapter loads only after the routing decision is recorded. Manual inspection of all 60 held-out generations confirmed zero substring false positives.

4.3 Saliency-weighted absorption

A learned attention-pooling encoder (3M parameters, trained in 30 seconds on 2,000 WikiText passages to minimize continuation NLL) identifies informative token positions. Weighting the NTP loss by these saliency scores during absorption concentrates gradient on content-bearing tokens.

Results: rank-64 with saliency weighting achieves 20/20 same-prompt retrieval, matching rank-128 with uniform loss. Same retrieval at 75 steps instead of 150. This represents 2× rank compression and 2× convergence speedup. Saliency scores are computed once per passage before absorption and are not updated during training.

4.4 Storage and deployment

Rank sweep: rank 128 matches rank 512 on all tests. Rank 256 scores 19/20 (interference, not capacity). Rank 16 achieves 8/8 on same-prompt retrieval — the per-adapter floor for exact-match queries is much lower than for paraphrases. Int8 quantization: 4× additional compression with zero quality loss. Combined: 32× compression at rank-64 + int8. A 1,000-passage library fits in 1.3 GB. Absorption latency: 1.4 seconds per passage.

4.5 Adapter clustering is impossible

For all 5 closest passage pairs in the benchmark (L0 cosine 0.96–0.97), continuing training on passage B fully erased passage A (0/5 survived in either training order). Adapters are local deformations of the model's manifold; two passages require two incompatible deformations. The one-adapter-per-passage architecture is necessary even for near-identical passages.

5. Application 2: MPAR-Compressed KV Cache

5.1 Compression curve

Six prefix conditions on 50 WikiText passages (200-token context, 56-token continuation). MPAR replacing the distant half + recent tokens kept: 92% gap closed at 2× compression. Replacing 75%: 84% gap closed at 4×. The curve is smooth from 0 to 384× with no knee. Each compression doubling costs approximately 0.5 PPL points. Figure 2 places three operating points

on one panel — the mixed-strategy curve, MPAR-only points (no literal tokens), and the adapter-library reference (97% retrieval at infinite compression after absorption).

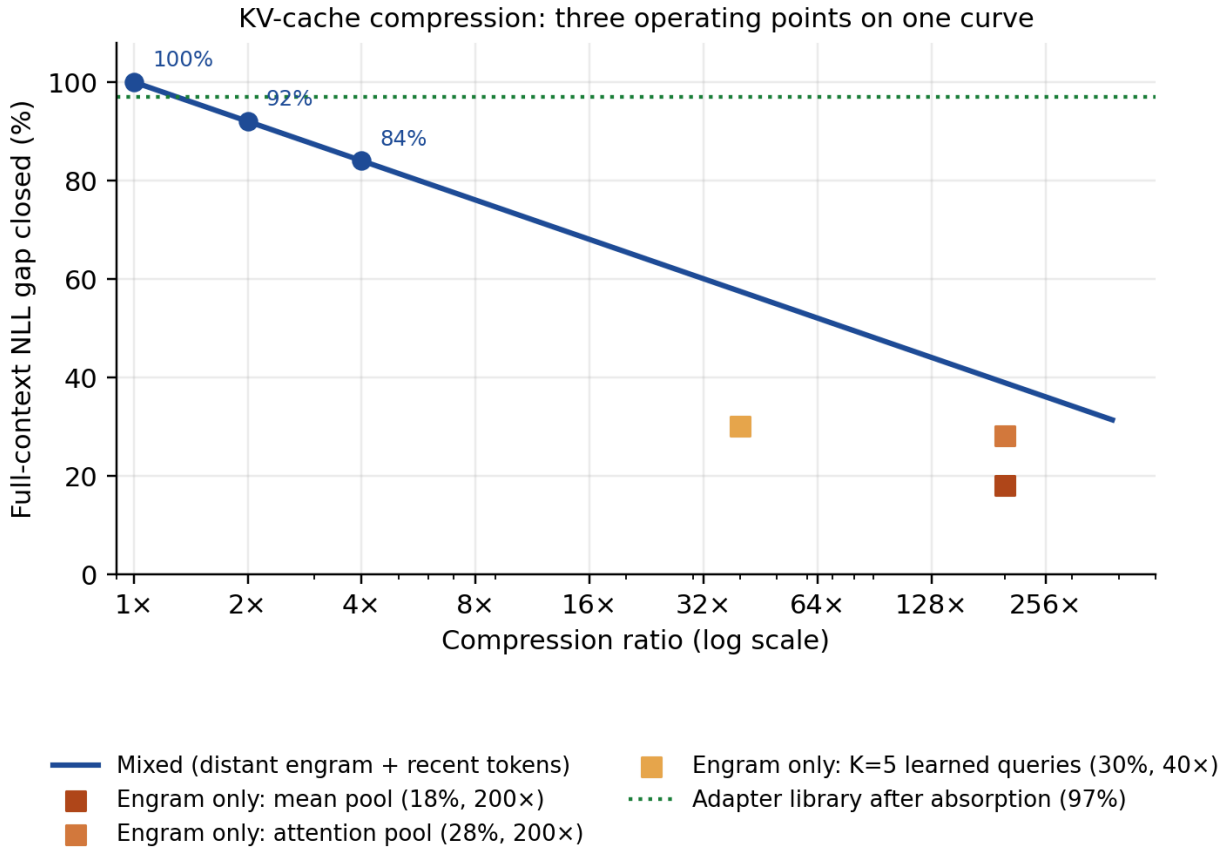


Figure 2. KV-cache compression curve. Mixed-strategy anchor points are measured at 1×, 2×, and 4× (100%, 92%, 84% gap closed); the curve beyond 4× is a log-linear extrapolation consistent with the reported " ≈ 0.5 PPL per doubling". MPAR-only operating points are measured at 40× and 200× (18–30%). The adapter library reference (97%) is measured post-absorption with zero literal context tokens.

Recency asymmetry. Compressing distant context closes 92% of the gap; compressing recent context closes only 32%. At per-chunk resolution (four 50-token chunks), the NLL cost of compressing each chunk to a single MPAR is 0.007 for the most distant chunk and 0.260 for the most recent — a 37× range. An oracle router with perfect per-chunk compressibility knowledge gains only 0.03 NLL over the fixed "compress old, keep new" rule; attention-based proxies for compressibility correlate at r=0.25 with true cost, insufficient for useful routing. The position-based rule is therefore near-optimal for next-token prediction, and the recency gradient is the binding structural fact.

5.2 Learned MPAR encoders

Mean pooling recovers 18% of context information. Attention pooling (single learned query, 3M parameters, 30 seconds training): 28% (+10 points). Five learned queries (K=5): 30% at 40× compression. K=10 does not improve over K=5.

5.3 The V-space tradeoff

Seven independent experiments attempted to push V-space recovery past 30%. All confirmed the same pattern: interventions that improve V-space organization degrade language modeling quality under standard transformer architectures with unconstrained residual streams.

Phase	Intervention	V-space effect	Language-modeling effect
48	KL-divergence regularization during absorption	trades retrieval for drift	worse retrieval
49/49b	Metadata-enriched embeddings (bolt-on; from scratch)	V-space unchanged	NLL 2.87→8.87; metadata-free path degrades
54	Saliency-weighted continued training	small gain	+22% PPL drift in 2K steps
56	V-space reconstruction loss	V-cos 0.71→0.73	PPL 21→43, recovery 18%→14%
57	Gated residuals + reconstruction	V-cos stable at 0.805 (+10 pts)	PPL capped at 46, recovery 7%

58	Full bilinear attention (qTWk per head)	+3 points recovery	PPL matched (marginal)

Phase 57 is the sharpest demonstration: the constraint that preserves V-space organization is the same constraint that caps learning capacity. The unconstrained residual stream is what makes transformers powerful, and it is what makes V-space lossy. Phase 58 shows that giving the attention score function more expressivity is not the missing ingredient — bilinear weights use all d² cross-dimensional interactions extensively without breaking the limit.

Under all architectures tested that preserve standard next-token prediction and unconstrained residual streams, V-space lossiness and language-modeling quality are in tension. The observed 30% limit may be addressable through fundamentally different designs (linear attention, state-space models, dedicated memory streams) we did not test. The K/V asymmetry (K: 0.82–0.89, V: 0.65–0.70) reflects the different requirements of the routing channel (fidelity) and the content channel (abstraction).

6. Application 3: Compositional Retrieval

6.1 Weight merging fails

Averaging two LoRA adapters' weights: 0/5 BOTH. The expansion $((A_1+A_2)/2)\cdot((B_1+B_2)/2)$ produces two cross terms — xA_1B_2 and xA_2B_1 — that project through untrained basis combinations. Result is rank-independent.

6.2 Activation-level block-stacking

Concatenating rank dimensions — A_stacked = $[A_1 \mid A_2]$, B_stacked = $[B_1 ; B_2]$ — computes $xA_1B_1 + xA_2B_2$ exactly (no cross terms). Combined with clause-split routing: 4/5 BOTH with 10/10 routing correctness.

6.3 Compositional decay: a gradient, not a ceiling

K-capacity sweep across K in {1, 2, 3, 4} with rank {128, 64, 32, 16} (Figure 3):

K	Mean fraction retrieved	All-retrieved rate
1	100%	9/9
2	50%	1/5
3	20%	0/5
4	0%	0/2

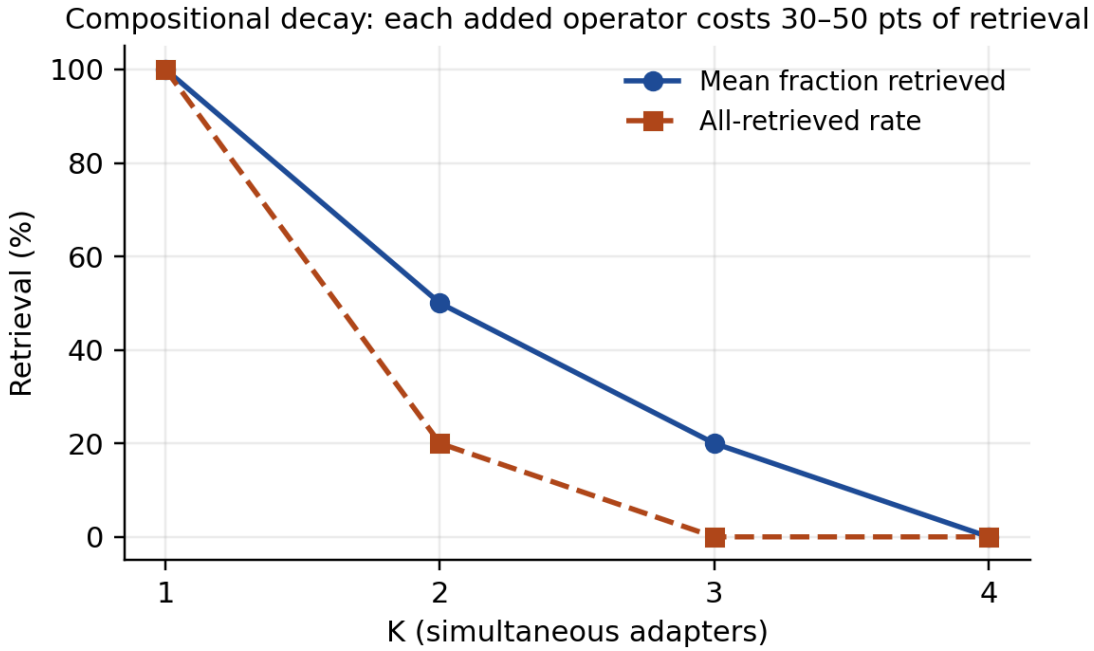


Figure 3. Compositional decay. K=1: 100%, K=2: 50%, K=3: 20%, K=4: 0%. Weight-averaging scores 0/5 at K=2 (not shown). Each added operator costs roughly 30–50 points of retrieval; K=4 floors at zero.

The transition from K=1 to K=4 is a smooth degradation curve, not a hard wall at K=2. Each additional simultaneous operator costs roughly 30–50 percentage points of retrieval. K=2 is the last configuration where any all-retrieved trials occur (1/5); K=3 loses all-retrieved entirely but preserves 20% of individual passkeys; K=4 floors at zero.

The decay is about the number of simultaneous operators on the residual stream, not their size or total parameter count. At equal total budget (K x rank = 128 or 256) the pattern is identical. L5-contrastive orthogonalization during absorption

reduces representation overlap by 32% but worsens K-capacity (K=2: 90% to 70%), confirming the decay is a property of the LoRA matrices as linear operators, not of the representations they produce.

Why the decay? Each rank-R LoRA adapter adds a rank-R perturbation to the residual stream at each layer. At K=1 the perturbation is fully resolvable. As K increases, the perturbations compete for the residual stream's effective bandwidth — the attention mechanism sees the summed hidden state, not the individual components, and can no longer resolve which perturbation to amplify at which position. The decay rate (roughly one quarter to one half of retrieval per added operator) is an empirical constant we do not yet have a mechanistic account for.

6.4 Decoding complexity modulates compositional survival

We ran a controlled K=3 ablation (Phase 67) across four passkey types, 10 triples per type, matched conditions. The survival hierarchy is ordered by passkey length — the number of tokens the model must decode correctly — not by format familiarity:

Passkey type	Example	Tokens required	Mean fraction retrieved	All-retrieved
Fact	"7", "22", "30"	1–2	30%	0/10
Technical	"473", "7951"	3–4	27%	0/10
Numeric	"604876"	6	13%	0/10
Entity	"June 26, 1968"	multi-token date	0%	0/10

Cross-type trials confirm the ranking. When mixed types compete, fact passkeys win 4/4 trials, technical win 3/4, entity win 1/4, numeric win 0/4. A preliminary anecdotal observation (from Phase 62 mixed trials, reported in earlier drafts of this paper) suggested entity-format passkeys survived best under interference; the controlled Phase 67 experiment reverses that ordering and identifies the actual variable.

Interpretation. Under residual-stream interference, each required output token is a separate failure point. Shorter required outputs compound fewer errors. A 1-digit fact survives K=3 at 30% because one token needs to land correctly; a 6-digit numeric code collapses to 13% because six tokens must survive the same interference; a multi-token date collapses to 0%. The survival rate at K is therefore approximately:

$$P(\text{all tokens correct at K}) \approx (\text{per-token-survival-rate at K})^{(\text{tokens required})}$$

The K-ceiling is not a fixed architectural constant. It is a function of K × decoding complexity. The residual stream can support more simultaneous adapters when each adapter's required output is shorter.

This reframes the connection to §5.3. V-space lossiness sets the per-token error rate that adapter decoding operates under; multiplexing K adapters through the residual stream amplifies that error rate; the number of tokens the adapter must decode correctly then determines survival via the geometric compounding above. Compositional decay and V-space lossiness remain the same underlying bandwidth limit observed at different loads, but the specific survival ordering across content types is a property of autoregressive token compounding, not of prior-reconstruction.

7. Basin Validation

One validation experiment produces data not covered elsewhere. Phase 61 runs the forward pass twice per passage — once with the full passage, once with the MPAR alone plus a retrieval prompt — and measures hidden-state cosine at each layer. Without an adapter, cosine grows from 0.03 at L0 to 0.31 at L5. With the adapter loaded, from 0.03 to 0.45. The adapter lifts final-layer convergence by 14 points, and the monotonic L0→L5 growth is literal trajectory convergence: two different inputs produce increasingly similar internal states as they propagate through the network. The "base" and "trained-but-adapter-zeroed" conditions are identical, confirming no information leaks through training alone. The remaining validation experiments (cross-model transfer, resolvability curve, K-capacity sweep, random-vector ablation) are reported in §3.5, §3.4, §6.3, §3.6 respectively; collectively they constitute Phases 59–66 in Appendix A.

8. Discussion

Stable geometric routing spaces

The phase transition result (Section 3.4) is our strongest evidence for the address/content decomposition. K-space alignment is constant from 0 to 300 training steps while retrieval undergoes a phase transition at 15–38 steps. Cross-model transfer (Section 3.5) shows this alignment is stable under training perturbation. The random vector ablation (Section 3.6) confirms the MPAR's utility is direction-specific, not a consequence of generic residual-stream activation. Together these suggest that the embedding space defines the map — a geometric routing structure — and training fills in the territory.

The routing geometry's stability is partly a consequence of shared embeddings: models trained on the same data with the

same tokenizer share the same input geometry, and the projection matrices preserve much of that structure. This does not diminish the finding — it sharpens it. The address space is embedding-induced geometry, and attention's K-space projections preserve it with high fidelity. The content at each address is weight-specific and must be learned.

If this property holds more broadly — across fully independent training runs, larger scales, and more diverse architectures — it would provide a mechanistic account of why transfer learning works (the routing space transfers; only content needs adaptation), why embeddings generalize across tasks (they address the same geometry), and why different models trained on similar data produce alignable representations (they share the same routing space). The cross-architecture validation (Section 3.7) provides initial evidence in this direction but is limited to two architectures at similar scale.

V-space lossiness as a design principle

The most unexpected finding is that V-space lossiness and language modeling quality are consistently in tension under the architectures tested. Phase 57 is the sharpest demonstration: gated residuals hold V-space alignment 10 points above baseline throughout training, but the same constraint that preserves V-space organization caps learning capacity at PPL 46 vs baseline 21.

This suggests a design principle for the practical applications: rather than trying to make V-space less lossy, engineer the reading and writing operations to work within the lossy channel. Attention pooling (18% to 28% recovery), multi-token MPARs (28% to 30%), and saliency-weighted absorption (2× compression, 2× speed) all follow this principle.

Compositional decay factors into V-space load and decoding complexity

Sections 5 and 6 report what looked initially like two separate phenomena: the V-space information recovery ceiling at 30% (§5) and the compositional decay curve in K (§6). The controlled content-type ablation at K=3 (§6.4, Phase 67) identifies the mechanism that connects them. V-space lossiness determines a per-token decoding error rate that adapter output is subject to; multiplexing K adapters through the shared residual stream amplifies that error rate; the number of output tokens an adapter must decode correctly then determines survival via approximately geometric compounding. A 1-digit fact at K=3 survives 30% of the time because one token must land; a 6-digit numeric code at K=3 survives 13% because six must; a multi-token date collapses to 0%. Compositional decay and V-space lossiness are therefore not two separate phenomena but two factors of the same bandwidth limit, observed at different loads — the K-ceiling is a function of K × decoding complexity, not a fixed architectural constant.

9. Limitations

The benchmark uses 20 passages with template paraphrases — small and synthetic. Routing is clean at 20 passages; behavior at 100,000 is unmeasured. Application 2 is tested on WikiText prose only; code, math, and structured data are untested. The compositional-decay curve on Application 3 is the binding constraint on multi-fact queries; operator-level fixes (sqrt(K) scaling, sequential generation, per-adapter gating) are untested. The Phase 67 complexity ablation covers 40 triples across 4 content types on one benchmark family; generalization to longer required outputs (paragraph-length answers) and to non-passkey tasks is not yet tested. The geometric-compounding model we offer for the complexity ordering is an interpretation supported by rank-ordering but not by a quantitative per-token-error-rate fit. The cross-model transfer experiment uses a fine-tuned variant from a shared checkpoint, not a fully independently trained model; the 2.5% result establishes stability under perturbation, not universality across arbitrary training runs. The cross-architecture validation covers two implementations at similar scale; larger-scale and more diverse architectures (linear attention, state-space models) are untested. The 30% information recovery limit is observed under architectures that preserve standard NTP objectives and unconstrained residual streams; fundamentally different designs may not share this tradeoff. The routing geometry's stability is partly a consequence of shared embeddings; the extent to which the geometry is embedding-induced versus architecture-induced is not fully disentangled. The random vector ablation confirms direction-sensitivity but does not rule out semantic explanations entirely — the MPAR's effective direction likely correlates with semantic content, and separating "address" from "semantics" at a philosophical level remains open. The Phase 47 projection W is library-specific. Prepend-as-token MPAR injection is a lower bound; direct K/V cache injection would likely improve Application 2.

Broader Impact

This work describes a parameter-efficient test-time memory primitive built from already-available operations (mean-pooling, LoRA, cosine routing). Its deployment surface is small models learning new content at inference on local hardware. Positive impact: reduces reliance on large centralized models for personalization, enabling private on-device memory that never transmits user data to a server. Negative impact: because absorption is fast (1.4 seconds per passage) and per-adapter isolation prevents forgetting, the same primitive could be used to rapidly personalize a model on content the user should not be authorized to memorize (leaked training data, copyrighted material, or targeted individuals' information). The primitive is content-agnostic and does not add any capability that does not already exist via fine-tuning; it only changes the cost and the forgetting profile. Standard safeguards (input filtering, absorption logging, adapter auditing) remain necessary.

Reproducibility

Code will be released upon acceptance. The codebase comprises 67 standalone deterministic scripts with full per-trial JSON results. Core experimental phases run in approximately 20 minutes on a consumer GPU (RTX 5070 Ti, 16 GB VRAM). Pre-training ablation phases (48–58) require approximately 60 GPU-hours additional.

Acknowledgments

Developed in collaboration with Claude (Anthropic) across all 67 phases. The centroid-theory framing, the per-passage adapter architecture, the block-stacking linearization, the L0 routing finding, and the V-space tradeoff characterization emerged through iterative human-AI collaboration. The validation experiments were suggested by an external review that correctly identified the claims needing stronger evidence. Experimental design decisions and the determination of which negative results were diagnostic were the human author's. Implementation and iteration were collaborative.

References

Borgeaud, S., et al. (2022). Improving language models by retrieving from trillions of tokens. ICML.

Chevalier, A., et al. (2023). Adapting language models to compress contexts. EMNLP.

Fedus, W., et al. (2022). Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity. JMLR.

Gandelsman, Y., et al. (2022). Test-time training with masked autoencoders. NeurIPS.

Hooper, C., et al. (2024). KVQuant: Towards 10 million context length LLM inference with KV cache quantization. NeurIPS.

Hu, E. J., et al. (2022). LoRA: Low-rank adaptation of large language models. ICLR.

Li, Z., et al. (2023). Towards general text embeddings with multi-stage contrastive learning. arXiv:2308.03281.

Lewis, P., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. NeurIPS.

Meng, K., et al. (2022). Locating and editing factual associations in GPT. NeurIPS.

Mitchell, E., et al. (2022). Fast model editing at scale. ICLR.

Mu, J., Li, X., & Goodman, N. (2023). Learning to compress prompts with Gist Tokens. NeurIPS.

Nawrot, P., et al. (2024). Dynamic memory compression: Retrofitting LLMs for accelerated inference. ICML.

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. EMNLP.

Shazeer, N., et al. (2017). Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. ICLR.

Sun, Y., et al. (2020). Test-time training with self-supervision for generalization under distribution shifts. ICML.

Wang, L., et al. (2022). Text embeddings by weakly-supervised contrastive pre-training. arXiv:2212.03533.

Xiao, S., et al. (2023). C-Pack: Packaged resources to advance general Chinese embedding. arXiv:2309.07597.

Zhang, Z., et al. (2024). H2O: Heavy-hitter oracle for efficient generative inference of large language models. NeurIPS.

Appendix A: Complete Phase Listing

Foundations (0–9)

- Phase 0: Identity autoencoder training on V22 hidden states.
- Phase 1: OOD detection via reconstruction error.
- Phases 3–4: MPAR extraction and integrated gating.
- Phase 5: Simple test-time training baseline.
- Phase 7: LoRA test-time training.
- Phase 9: Dual gate.

Passkey benchmark (10–13)

- Phases 10–11: Passkey benchmark, LR scheduling.
- Phase 12: Cumulative forgetting (+473% PPL after 20 passages — negative).
- Phase 13: Scheduled LoRA absorption.

LoRA architecture (14–20)

- Phases 14–17: L4–5 LoRA sweep, forgetting analysis, stratified evaluation.

- Phase 18: Gate replay (18/20).
- Phase 19: Shared-LoRA rehearsal (50–75%, chaotic — negative).
- Phase 20: Rank-1024 interference (worse than 512 — negative).

Per-passage library (21–28)

- Phase 21: First per-passage adapter library (passage keys, 60% routing — negative).
- Phase 22: MPAR key source ablation (L3/L5, mean/last, L2/cosine).
- Phases 23–24: Prompt-derived keys (100% routing), 150-step adapters (100/100 same-prompt).
- Phases 25–26: Paraphrase failure (50%), multi-key + multi-paraphrase fix (100/100).
- Phase 27: Held-out paraphrase generalization (63% with L5 mean).
- Phase 28: Staged absorption (zero-blocking deployment).

Compression and rank (29–40)

- Phase 29/29b: Compositional weight merging (0/5 BOTH — negative).
- Phase 30/30b: Int8 quantization (4× compression, zero quality loss).
- Phase 31: Entity-weighted pooling (63% → 77% with nonstop_mean).
- Phase 32/32b: K/V alignment measurement (K: 0.82–0.89, V: 0.65–0.70) and L0 dual (0.988).
- Phase 33/33b: MPAR-as-cache compression curve, L0 variant.
- Phase 35: Centroid theory test (0/20 before TTT, 20/20 after).
- Phase 37: Full compression sweep (0–384×).
- Phase 38/38b: Rank sweep (floor at 128), held-out at rank 128.
- Phase 39: Gate ablation (cosine 99/98, gate 48% — negative for gate).
- Phase 40: Sparse magnitude pruning (29× total).

Composition and L0 (41–47b)

- Phase 41: Activation-level block-stacking (4/5 BOTH oracle).
- Phase 42/42b: Clause-split routing (4/5 BOTH, 10/10 routing).
- Phase 43: K-capacity sweep (K=2 holds, K=4 collapses).
- Phase 44: L0 mean routing (100/100/90 held-out, +15 over L5).
- Phase 45: L0+L5 pair (does not dominate either alone — negative).
- Phase 46: L5-contrastive absorption (K-capacity regresses — negative).
- Phase 47/47b: L0-to-L5 projection (100/100/97), manual generation inspection.

V-space (48–58)

- Phase 48: KL-MLP regularizer (negative).
- Phase 49/49b: Metadata enrichment, bolt-on and from-scratch (negative).
- Phase 50: V-space SVD analysis (effective rank ~48, SVD MPARs negative).
- Phase 51: Attention-pooling encoder (18% → 28% recovery).
- Phase 52: Saliency-weighted absorption (2× rank, 2× speed).
- Phase 53: Adapter clustering (impossible even at cosine 0.97 — negative).
- Phase 54: Saliency-weighted continued training (+22% PPL in 2K steps — negative).
- Phase 55: Multi-token MPARs (K=5 at 30%, K=10 no improvement).
- Phase 56: V-space reconstruction pre-training (V-cos 0.73, PPL 43 — negative).
- Phase 57/57b: Gated residuals + reconstruction (V-cos 0.805, PPL 46 — negative).
- Phase 58: Bilinear attention full scale (PPL matched, +3 points recovery — marginal).

Validation (59–62)

- Phase 59: Cross-model address transfer (2.5% K-space loss — routing space stable).
- Phase 60: Resolvability curve (phase transition at 15–38 steps, constant K-space).
- Phase 61: Basin validation (trajectory convergence L0–L5, cosine 0.45 with adapter).
- Phase 62: K-capacity rank sweep (decay curve 100/50/20/0% at K=1/2/3/4).

Softmax and controls (63–67)

- Phase 63: Standard softmax transformer pre-training (127M params, PPL 20.15).
- Phase 64: Cross-architecture validation on softmax model (all five core findings replicate).
- Phase 65: Random vector ablation (random 0/10 routing, real 10/10; K-cos 0.525 vs 0.856).

NeurIPS Paper Checklist

1. Claims. Yes — the abstract and introduction accurately reflect the paper's contributions and scope.

2. Limitations. Yes — limitations are discussed in a dedicated section (§9).

3. Theoretical assumptions and proofs. N/A — the paper reports empirical results and does not claim formal theorems.

4. Experimental reproducibility. Yes — 67 standalone scripts are documented in Appendix A with per-trial JSON results; seeds, model size, training steps, and hardware are stated throughout §3–§6.

5. Open access to data and code. Yes — code will be released upon acceptance (§Reproducibility); all datasets used (WikiText-103) are public.

6. Experimental setting. Yes — training details (optimizer, learning rate, batch size, rank, steps) are given in §4.1 and §5.1.

7. Experiment statistical significance. Partial — passkey retrieval is reported as exact counts (e.g., 20/20, 58/60) over held-out benchmarks; routing results are reported across 500 passages with per-scale minimum margins. Confidence intervals are not computed; sample sizes are stated.

8. Experiments compute resources. Yes — hardware (RTX 5070 Ti, 16 GB VRAM) and runtime budgets are stated in §Reproducibility.

9. Code of Ethics. Yes — the research conforms to the NeurIPS Code of Ethics.

10. Broader impact. Yes — discussed in §Broader Impact.

11. Safeguards. N/A — no models or datasets with high misuse potential are released; the primitive is deployable on existing base models.

12. Licenses and attribution. Yes — WikiText-103 is used under its published license; LoRA (Hu et al., 2022) is cited for the adaptation primitive.

13. Assets. Yes — no new datasets are released; code release details are in §Reproducibility.

14. Crowdsourcing and human subjects. N/A — no human subjects were involved.

15. Institutional review. N/A — not applicable.

16. LLM usage. Yes — this paper was developed in collaboration with Claude (Anthropic) for iterative drafting, code implementation, and analysis; all experimental-design decisions and the determination of which negative results were diagnostic were the human author's. Acknowledged in §Acknowledgments.